

Datadog Confluent Platform Integration

Datadog Confluent Platform Integration

Overview

Datadog ships a first-party **Confluent Platform** integration (check name: `confluent_platform`).

It collects JMX metrics from the following CFK/Confluent components:

Component	JMX Port (CFK default)
Kafka Broker	7203
Kafka Connect	7203
Schema Registry	7203
ksqlDB Server	7203
REST Proxy	7203
Replicator	7203
Kafka Streams	7203

Minimum Agent version required: **7.19.0**

Official docs: https://docs.datadoghq.com/integrations/confluent_platform/

How It Differs from the OpenMetrics Approach Used Here

Dimension	This workspace (OpenMetrics)	Confluent Platform integration (JMX)
Collection method	HTTP scrape of <code>:7778/metrics</code> (Prometheus exposition)	JMX pull from <code>:7203</code>
Setup		

Dimension	This workspace (OpenMetrics)	Confluent Platform integration (JMX)
	Manual metric list in pod annotations	<code>collect_default_metrics: true</code> covers all mapped MBeans
Metric namespace	Custom <code>cfk.*</code>	Standard <code>confluent.*</code>
Out-of-the-box dashboards	None — custom-built	Provided by Datadog, kept up to date
Recommended monitors/alerts	None built-in	Ships with suggested alert conditions
JMX-only MBeans	Not accessible	Fully accessible
Tag enrichment	Manual via pod labels	Auto-enriched from JMX MBean object names
Maintenance	You own it — new metrics need annotation updates	Datadog updates the check definition
Agent Helm flag required	None (OpenMetrics is always on)	<code>datadog.jmxfetch.enabled=true</code>

What Metrics It Brings (Key Categories)

Kafka Connect

- `confluent.kafka.connect.worker.connector_running_task_count`
- `confluent.kafka.connect.worker.connector_failed_task_count`
- `confluent.kafka.connect.worker.connector_paused_task_count`
- `confluent.kafka.connect.connector_task.running_ratio`
- `confluent.kafka.connect.source_task.source_record_write_rate`
- `confluent.kafka.connect.sink_task.sink_record_active_count_avg`
- `confluent.kafka.connect.sink_task.put_batch_avg_time_ms`
- `confluent.kafka.connect.task_error.total_record_errors`
- `confluent.kafka.connect.task_error.total_retries`
- `confluent.kafka.connect.task_error.deadletterqueue_produce_requests`
- `confluent.kafka.connect.worker_rebalance.rebalancing`

Kafka Broker

- `confluent.kafka.server.replica_manager.under_replicated_partitions`
- `confluent.kafka.server.replica_manager.isr_shrinks_per_sec.rate`
- `confluent.kafka.server.topic.bytes_in_per_sec.rate`
- `confluent.kafka.server.topic.messages_in_per_sec.rate`
- `confluent.kafka.controller.active_controller_count`
- `confluent.kafka.controller.offline_partitions_count`

Schema Registry

- `confluent.kafka.schema.registry.registered_count`
- `confluent.kafka.schema.registry.master_slave_role.master_slave_role`
- `confluent.kafka.schema.registry.jetty.connections_active`

ksqlDB

- `confluent.ksql.query_stats.messages_consumed_per_sec`
- `confluent.ksql.query_stats.num_active_queries`
- `confluent.ksql.query_stats.error_rate`
- `confluent.ksql.pull_query_metrics.pull_query_requests_rate`

Service Check

- `confluent.can_connect` — returns OK, WARNING, or CRITICAL per component instance.

Steps to Enable in Datadog SaaS

Step 1 — Enable the tile in Datadog UI

1. Go to **Integrations** → **Integrations** in Datadog.
2. Search for **Confluent Platform**.
3. Click **Install Integration** (no API credential is needed; this activates the tile and OOB dashboards).

Step 2 — Enable JMX collection in the Datadog Agent (Helm)

The minimal agent deployment in this workspace has JMX disabled. Upgrade Helm to enable it:

```
API_KEY='<your-datadog-api-key>'
```

```
helm upgrade --install datadog datadog/datadog \
  -n datadog-agent \
  --set datadog.apiKey="$API_KEY" \
  --set datadog.site=us5.datadoghq.com \
  --set clusterAgent.enabled=false \
  --set datadog.logs.enabled=false \
  --set datadog.apm.enabled=false \
  --set datadog.processAgent.enabled=false \
  --set datadog.jmxfetch.enabled=true # required for
  confluent platform check
```

Step 3 — Replace OpenMetrics annotations with JMX autodiscovery annotations

For each CFK pod resource, replace the current openmetrics annotations with confluent_platform JMX annotations.

connect.yaml (and similarly for kafka.yaml, schemaregistry.yaml, ksqldb.yaml, kafkarestproxy.yaml, kraftcontroller.yaml, controlcenter.yaml):

```
# REMOVE these:
ad.datadoghq.com/connect.check_names: '["openmetrics"]'
ad.datadoghq.com/connect.init_configs: '[]'
ad.datadoghq.com/connect.instances:
  '[{"openmetrics_endpoint":"http://%%host%%:7778/
    metrics", ...}]'

# ADD these:
ad.datadoghq.com/connect.check_names: '["confluent_platform"]'
ad.datadoghq.com/connect.init_configs: '[{"is_jmx": true,
  "collect_default_metrics": true}]'
ad.datadoghq.com/connect.instances: '[{
  "host": "%%host%%",
  "port": 7203,
  "name": "connect_instance",
  "tags": ["component:connect", "env:local-poc", "service:kafka-
    connect"]
}]'
```

Note: The port value must match CFK's JMX port, which is 7203 by default (set in spec.metrics.jmxPort or the CFK operator default).

Step 4 — Apply the updated resources and verify

```
kubectl apply -f connect.yaml
kubectl apply -f kafka.yaml
kubectl apply -f schemaregistry.yaml
kubectl apply -f ksqldb.yaml
kubectl apply -f kafkarestproxy.yaml
```

```
# Verify the Datadog agent picked up the JMX check
kubectl exec -n datadog-agent \
  $(kubectl get pod -n datadog-agent -l app=datadog -o
    jsonpath='{.items[0].metadata.name}') \
  -- agent status | grep -A 10 "confluent_platform"
```

Expected output:

```
JMXFetch
=====
  Initialized checks
  =====
    confluent_platform
      instance_name : confluent_platform-<host>-7203
```

```
metric_count : 26
status       : OK
```

Step 5 — Validate metrics in Datadog

1. Go to **Metrics** → **Explorer**.
2. Search for
`confluent.kafka.connect.worker.connector_running_task_count`.
3. Confirm data is flowing with the correct tags.

Step 6 — Use the out-of-the-box dashboard

1. Go to **Dashboards** → **Dashboard List**.
2. Search for **Confluent Platform**.
3. Clone the OOB dashboard if you want to customise it without losing the original.

Dashboard Metric Name Migration

If you want to keep your existing custom dashboards and switch to the Confluent Platform integration, all metric names must change. Example mapping:

Current (cfk.* OpenMetrics)	Confluent P
cfk.kafka_connect_connect_worker_metrics_connector_running_task_count	confluent.
cfk.kafka_connect_sink_task_metrics_sink_record_active_count_avg	confluent.
cfk.kafka_connect_sink_task_metrics_put_batch_avg_time_ms	confluent.
cfk.kafka_connect_task_error_metrics_total_record_errors	confluent.
cfk.kafka_connect_task_error_metrics_deadletterqueue_produce_requests	confluent.
cfk.java_lang_operatingsystem_processcpuload	No direct eq integration
cfk.java_lang_memory_heapmemoryusage_used	No direct eq

Why the OpenMetrics Approach Was Used Here

- The Confluent Platform JMX check requires the Agent to open a direct TCP connection to `:7203` on each CFK pod. In Docker Desktop Kubernetes this works, but the setup is more complex in network-restricted or multi-namespace environments.
- OpenMetrics scraping is fully declarative via pod annotations — no agent config file is needed.
- Custom `cfk.*` metric namespace gave precise control over what is ingested and at what cost.
- The JMX approach is better suited for production environments where you want OOB dashboards, alerts, and lower maintenance overhead.